



ЗВІТ

Лабораторна робота № 2

Дисципліна: Базы даних та інформаційні системи

Тема: Проектування та створення бази даних

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

Хід роботи

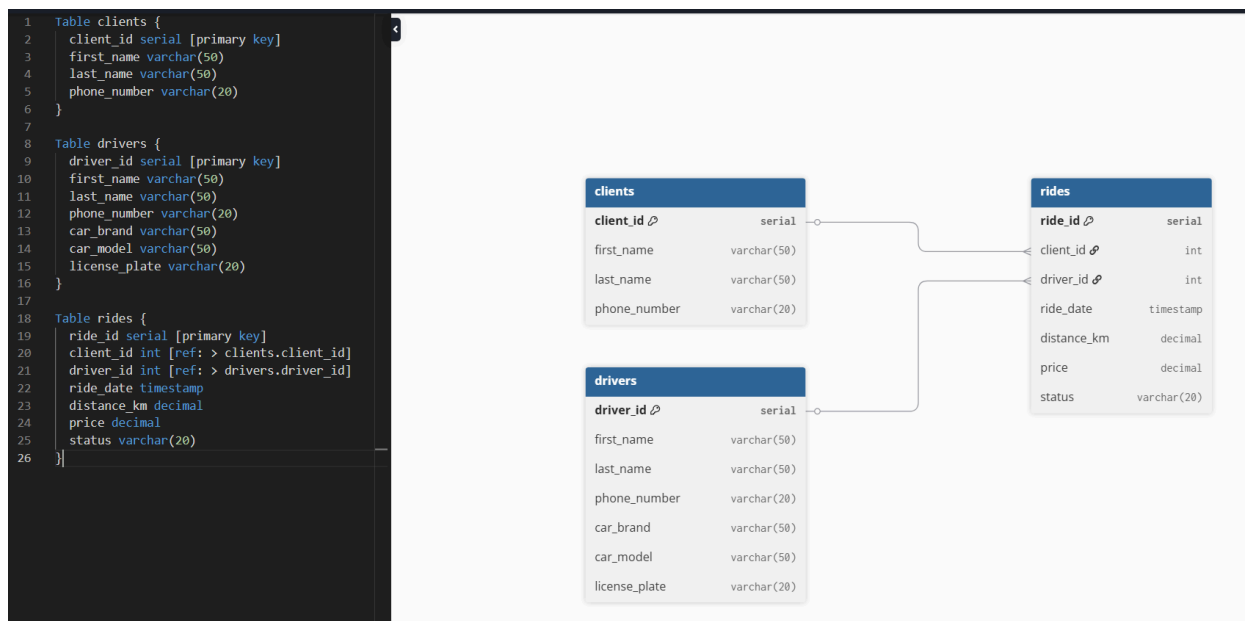
Варіант №19: База даних для служби таксі.

1. Опис бізнес-процесу (формат .txt)

Виконати опис бізнес-процесу, який буде реалізовано, відповідно до отриманої теми. Структуровано викласти ключові аспекти роботи системи, включаючи основні сутності та взаємодію між ними.

2. Створення ER-діаграми (формат .pdf)

Відповідно до описаного бізнес-процесу створити ER-діаграму бази даних за допомогою dbdiagram.io. Зберегти діаграму у форматі PDF.



3. Реалізація бази даних та користувачів (формат .sql) Створити базу даних у PostgreSQL або MySQL. Створити кілька користувачів з різними рівнями доступу (адміністратор, модератор, звичайний користувач).

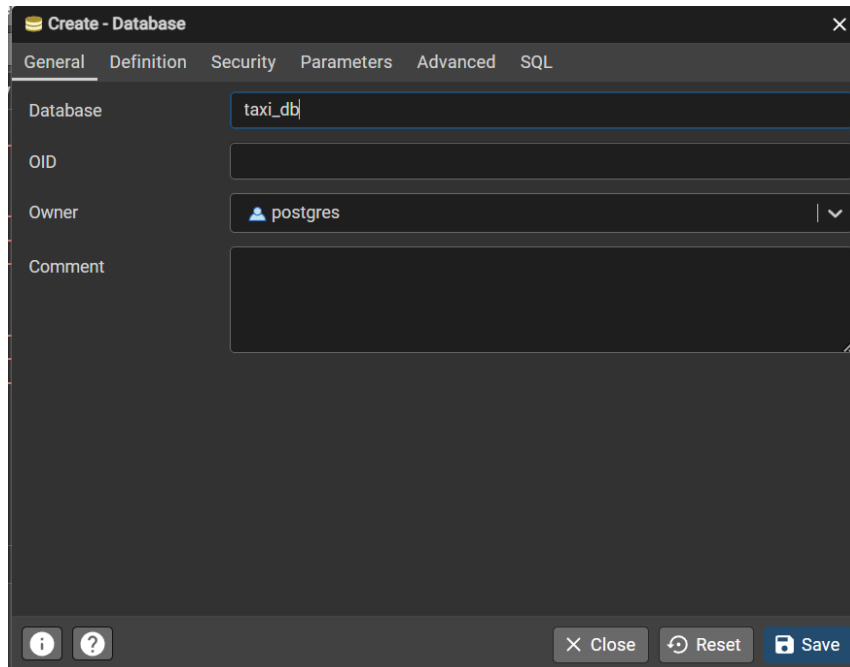


Рисунок 3.1. Створив базу даних **taxi_db** у PostgreSQL.

```
Query  Query History
1  -- 1. Створення бази даних
2  CREATE DATABASE taxi_db;
3
4  -- 2. Створення користувачів із паролями
5  CREATE USER taxi_admin WITH PASSWORD 'admin123';
6  CREATE USER taxi_moderator WITH PASSWORD 'mod123';
7  CREATE USER taxi_user WITH PASSWORD 'user123';
8
9  -- 3. Надання базових прав доступу до бази
10 GRANT ALL PRIVILEGES ON DATABASE taxi_db TO taxi_admin;
11 GRANT CONNECT ON DATABASE taxi_db TO taxi_moderator;
12 GRANT CONNECT ON DATABASE taxi_db TO taxi_user;
```

Data Output Messages Notifications

Рисунок 3.2. Створив користувачів з різними рівнями доступу.

4. Створення таблиць бази даних (формат .sql).

```
postgres
taxi_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
Login/Group Roles
Database Spaces

1 CREATE TABLE clients (
2   client_id SERIAL PRIMARY KEY,
3   first_name VARCHAR(50) NOT NULL,
4   last_name VARCHAR(50) NOT NULL,
5   phone_number VARCHAR(20)
6 );
7
8 CREATE TABLE drivers (
9   driver_id SERIAL PRIMARY KEY,
10  first_name VARCHAR(50) NOT NULL,
11  last_name VARCHAR(50) NOT NULL,
12  phone_number VARCHAR(20),
13  car_brand VARCHAR(50),
14  car_model VARCHAR(50),
15  license_plate VARCHAR(20)
16 );
17
18 CREATE TABLE rides (
19   ride_id SERIAL PRIMARY KEY,
20   client_id INT NOT NULL,
21   driver_id INT NOT NULL,
22   ride_date TIMESTAMP,
23   distance_km DECIMAL(5, 2),
24   price DECIMAL(7, 2),
25   status VARCHAR(20),
26   FOREIGN KEY (client_id) REFERENCES clients(client_id) ON DELETE CASCADE,
27   FOREIGN KEY (driver_id) REFERENCES drivers(driver_id) ON DELETE CASCADE
28 );

Data Output Messages Notifications
CREATE TABLE
```

5. Заповнення таблиць тестовими даними (формат .sql).

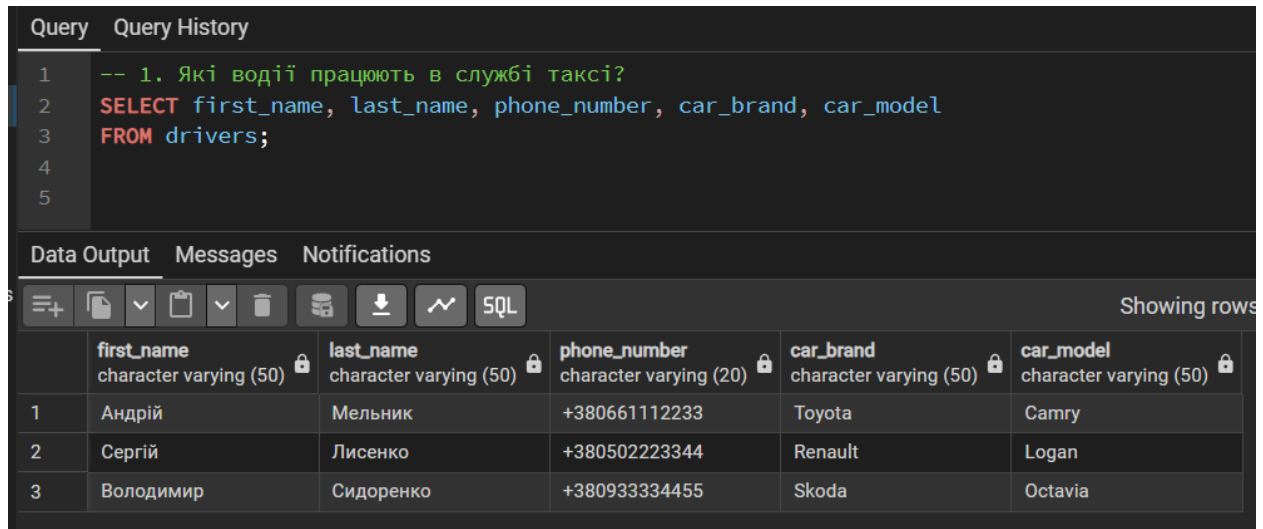
```
Query Query History

1 -- Додавання клієнтів
2 INSERT INTO clients (first_name, last_name, phone_number) VALUES
3 ('Олександр', 'Коваленко', '+380501234567'),
4 ('Марія', 'Шевченко', '+380671234567'),
5 ('Іван', 'Бойко', '+380631234567'),
6 ('Олена', 'Ткаченко', '+380991234567'),
7 ('Дмитро', 'Кравченко', '+380971234567');
8
9 -- Додавання водіїв
10 INSERT INTO drivers (first_name, last_name, phone_number, car_brand, car_model, license_plate) VALUES
11 ('Андрій', 'Мельник', '+380661112233', 'Toyota', 'Camry', 'KA1234BX'),
12 ('Сергій', 'Лисенко', '+380502223344', 'Renault', 'Logan', 'AI5678CX'),
13 ('Володимир', 'Сидоренко', '+380933334455', 'Skoda', 'Octavia', 'CE9012AA');
14
15 -- Додавання поїздок
16 INSERT INTO rides (client_id, driver_id, ride_date, distance_km, price, status) VALUES
17 (1, 1, '2026-05-01 08:30:00', 12.5, 250.00, 'completed'),
18 (2, 2, '2026-05-01 09:15:00', 5.2, 120.00, 'completed'),
19 (3, 1, '2026-05-02 14:00:00', 8.0, 160.00, 'completed'),
20 (4, 3, '2026-05-02 18:45:00', 22.4, 450.00, 'completed'),
21 (5, 2, '2026-05-03 10:20:00', 3.1, 85.00, 'cancelled'),
22 (1, 3, '2026-05-03 22:10:00', 15.0, 300.00, 'completed'),
23 (2, 1, '2026-05-04 07:50:00', 9.5, 190.00, 'completed'),
24 (3, 2, '2026-05-04 13:30:00', 4.0, 100.00, 'completed'),
25 (4, 1, '2026-05-05 19:00:00', 18.2, 380.00, 'completed'),
26 (5, 3, '2026-05-06 11:15:00', 7.7, 150.00, 'completed');

Data Output Messages Notifications
INSERT 0 10
```

6. Виконання SQL-запитів SELECT (формат .sql). Для аналізу даних бази «Служба таксі» було написано 10 SQL-запитів, які охоплюють вибірку з умовою (WHERE), сортування (ORDER BY), групування (GROUP BY + HAVING), об'єднання таблиць (JOIN) та використання агрегатних функцій.

Запит 1. Виведення інформації про всіх водіїв служби таксі.



The screenshot shows a SQL query editor with a query window and a results window. The query window contains the following SQL code:

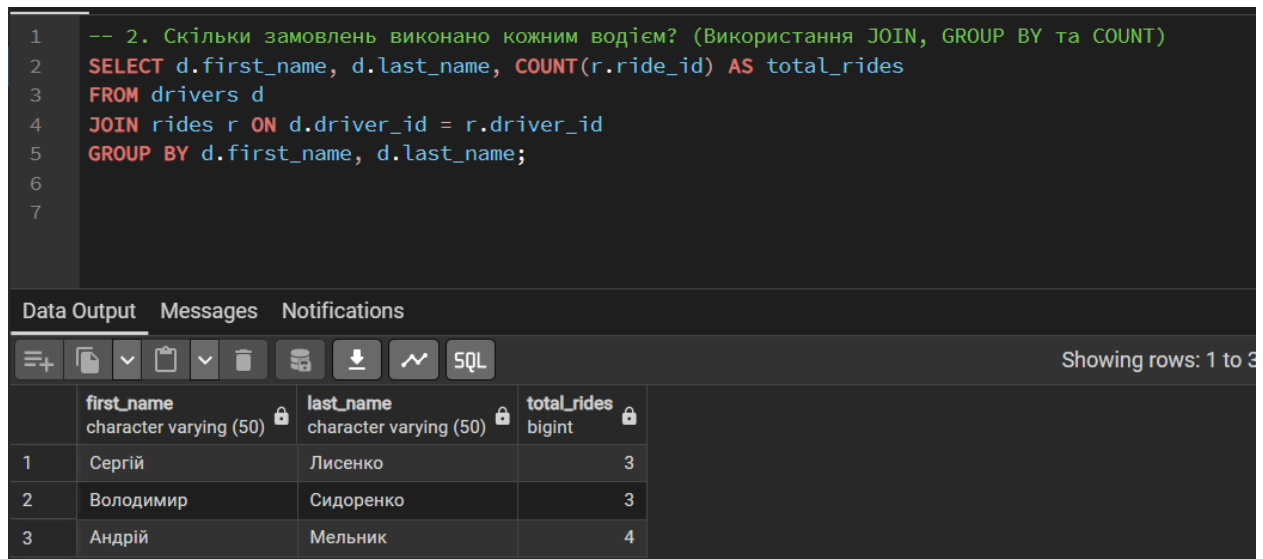
```
1 -- 1. Які водії працюють в службі таксі?
2 SELECT first_name, last_name, phone_number, car_brand, car_model
3 FROM drivers;
4
5
```

The results window shows the output of the query, displaying columns: first_name, last_name, phone_number, car_brand, and car_model. The results are as follows:

	first_name character varying (50)	last_name character varying (50)	phone_number character varying (20)	car_brand character varying (50)	car_model character varying (50)
1	Андрій	Мельник	+380661112233	Toyota	Camry
2	Сергій	Лисенко	+380502223344	Renault	Logan
3	Володимир	Сидоренко	+380933334455	Skoda	Octavia

Рисунок 6.1. Результат виконання простого запити SELECT.

Запит 2. Підрахунок кількості виконаних замовлень для кожного водія.



The screenshot shows a SQL query editor with a query window and a results window. The query window contains the following SQL code:

```
1 -- 2. Скільки замовлень виконано кожним водієм? (Використання JOIN, GROUP BY та COUNT)
2 SELECT d.first_name, d.last_name, COUNT(r.ride_id) AS total_rides
3 FROM drivers d
4 JOIN rides r ON d.driver_id = r.driver_id
5 GROUP BY d.first_name, d.last_name;
6
7
```

The results window shows the output of the query, displaying columns: first_name, last_name, and total_rides. The results are as follows:

	first_name character varying (50)	last_name character varying (50)	total_rides bigint
1	Сергій	Лисенко	3
2	Володимир	Сидоренко	3
3	Андрій	Мельник	4

Рисунок 6.2. Результат запити з використанням JOIN, GROUP BY та функції COUNT.

Запит 3. Розрахунок середньої відстані по всіх поїздках.

The screenshot shows a SQL query editor with a dark theme. The 'Query' tab is active, displaying a query to calculate the average distance of all rides. The query is as follows:

```
1 -- 3. Яка середня відстань по кожному замовленню? (Використання AVG)
2 SELECT ROUND(AVG(distance_km), 2) AS avg_distance_km
3 FROM rides;
```

Below the query editor, the 'Data Output' tab is active, showing the result of the query. The result is a single row with the column 'avg_distance_km' and the value 10.56.

	avg_distance_km numeric
1	10.56

Рисунок 6.3. Результат запиту з використанням агрегатної функції AVG.

Запит 4. Пошук клієнтів, які зробили більше одного замовлення.

The screenshot shows a SQL query editor with a dark theme. The 'Query' tab is active, displaying a query to find clients who have made more than one ride. The query is as follows:

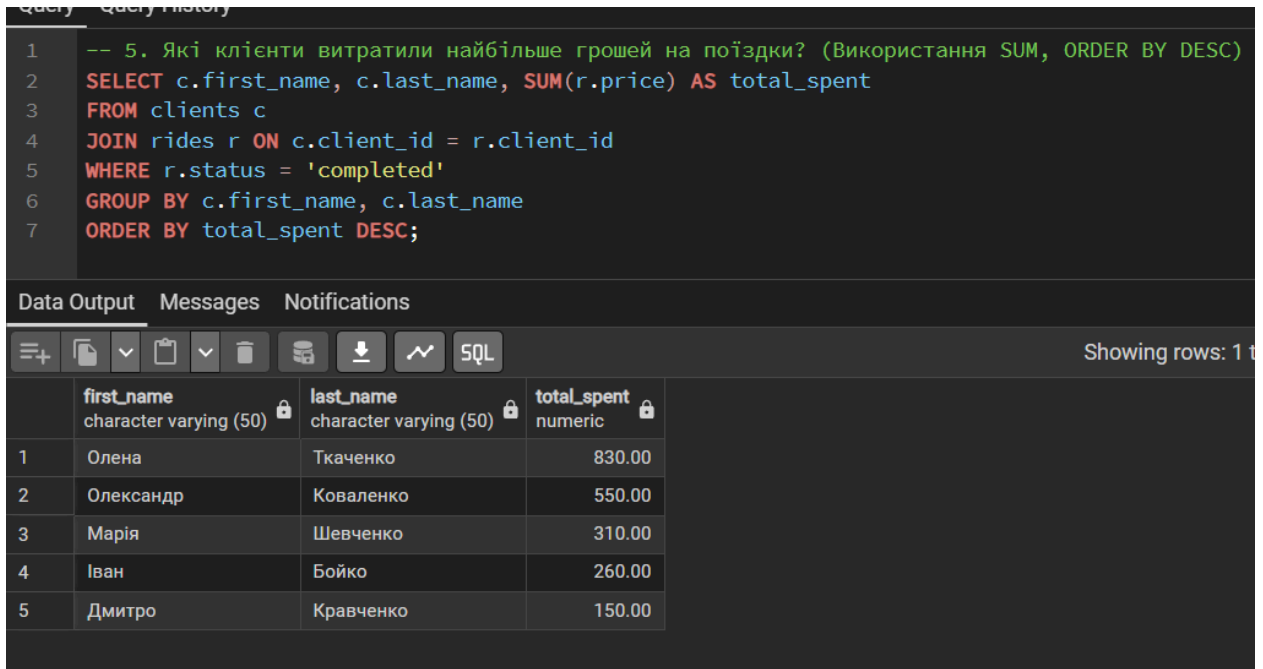
```
1 -- 4. Скільки клієнтів зробили більше одного замовлення? (Використання GROUP BY, HAVING)
2 SELECT c.first_name, c.last_name, COUNT(r.ride_id) AS ride_count
3 FROM clients c
4 JOIN rides r ON c.client_id = r.client_id
5 GROUP BY c.first_name, c.last_name
6 HAVING COUNT(r.ride_id) > 1;
```

Below the query editor, the 'Data Output' tab is active, showing the result of the query. The result is a table with 5 rows, showing the first name, last name, and ride count for each client.

	first_name character varying (50)	last_name character varying (50)	ride_count bigint
1	Іван	Бойко	2
2	Марія	Шевченко	2
3	Олександр	Коваленко	2
4	Олена	Ткаченко	2
5	Дмитро	Кравченко	2

Рисунок 6.4. Результат запиту з використанням GROUP BY та умови HAVING.

Запит 5. Визначення клієнтів, які витратили найбільше грошей на поїздки.

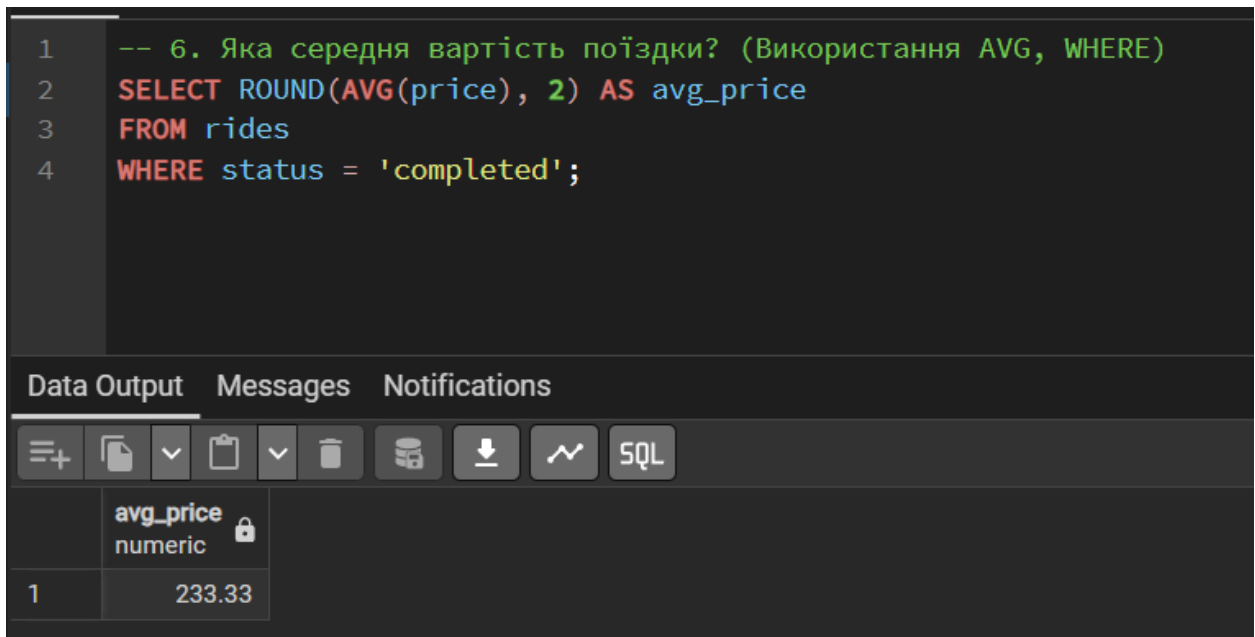


```
1  -- 5. Які клієнти витратили найбільше грошей на поїздки? (Використання SUM, ORDER BY DESC)
2  SELECT c.first_name, c.last_name, SUM(r.price) AS total_spent
3  FROM clients c
4  JOIN rides r ON c.client_id = r.client_id
5  WHERE r.status = 'completed'
6  GROUP BY c.first_name, c.last_name
7  ORDER BY total_spent DESC;
```

	first_name character varying (50)	last_name character varying (50)	total_spent numeric
1	Олена	Ткаченко	830.00
2	Олександр	Коваленко	550.00
3	Марія	Шевченко	310.00
4	Іван	Бойко	260.00
5	Дмитро	Кравченко	150.00

Рисунок 6.5. Результат запиту з використанням функції SUM та сортування ORDER BY DESC.

Запит 6. Обчислення середньої вартості успішно виконаної поїздки.



```
1  -- 6. Яка середня вартість поїздки? (Використання AVG, WHERE)
2  SELECT ROUND(AVG(price), 2) AS avg_price
3  FROM rides
4  WHERE status = 'completed';
```

	avg_price numeric
1	233.33

Рисунок 6.6. Результат запиту з використанням AVG та умови WHERE.

Запит 7. Фільтрація поїздки, здійснених у нічний час (з 22:00 до 06:00).

Query		Query History			
1	-- 7. Які замовлення здійснені в нічний час (після 22:00 або до 06:00)? (Використання WHERE)				
2	SELECT ride_id, ride_date, distance_km, price				
3	FROM rides				
4	WHERE EXTRACT(HOUR FROM ride_date) >= 22 OR EXTRACT(HOUR FROM ride_date) < 6;				

Data Output		Messages	Notifications	
Showing rows: 1 to				
	ride_id [PK] integer	ride_date timestamp without time zone	distance_km numeric (5,2)	price numeric (7,2)
1	6	2026-05-03 22:10:00	15.00	300.00

Рисунок 6.7. Результат запиту з використанням складних умов фільтрації WHERE.

Запит 8. Підрахунок унікальних автомобілів (водіїв), що виконували поїздки.

1	-- 8. Скільки унікальних автомобілів (водіїв) виконували успішні поїздки?		
2	SELECT COUNT(DISTINCT driver_id) AS active_cars		
3	FROM rides		
4	WHERE status = 'completed';		

Data Output		Messages	Notifications
	active_cars bigint		
1	3		

Рисунок 6.8. Результат запиту з використанням COUNT(DISTINCT).

Запит 9. Виведення детальної інформації про скасовані замовлення.

The screenshot shows a SQL query editor with a dark theme. The 'Query' tab is active, displaying a SQL query. Below the query, the 'Data Output' tab shows the results of the query in a table format. The table has five columns: ride_id, client_name, driver_name, and ride_date. The first row of data shows ride_id 5, client_name 'Дмитро', driver_name 'Сергій', and ride_date '2026-05-03 10:20:00'.

```
1 -- 9. Які записи містять інформацію про скасування замовлення? (Використання JOIN, WHERE)
2 SELECT r.ride_id, c.first_name AS client_name, d.first_name AS driver_name, r.ride_date
3 FROM rides r
4 JOIN clients c ON r.client_id = c.client_id
5 JOIN drivers d ON r.driver_id = d.driver_id
6 WHERE r.status = 'cancelled';
```

	ride_id integer	client_name character varying (50)	driver_name character varying (50)	ride_date timestamp without time zone
1	5	Дмитро	Сергій	2026-05-03 10:20:00

Рисунок 6.9. Результат запиту на об'єднання трьох таблиць за допомогою JOIN.

Запит 10. Підрахунок кількості поїздок за визначений період часу.

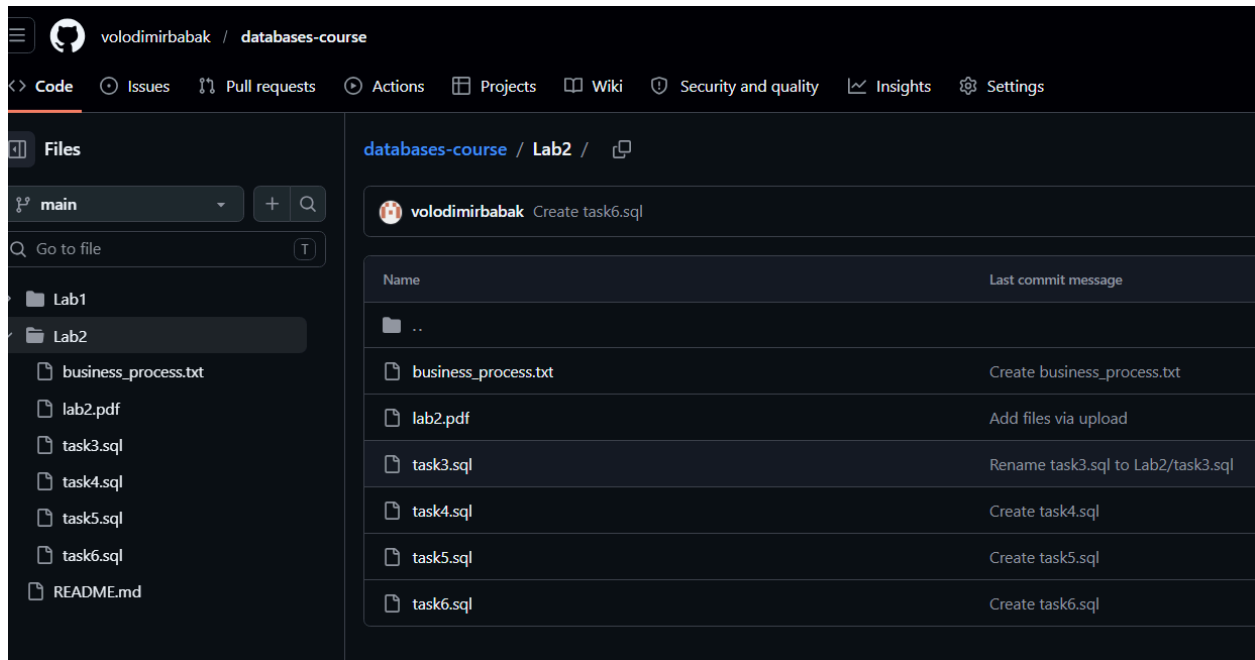
The screenshot shows a SQL query editor with a dark theme. The 'Query' tab is active, displaying a SQL query. Below the query, the 'Data Output' tab shows the results of the query in a table format. The table has one column: rides_in_period. The first row of data shows rides_in_period 6.

```
1 -- 10. Скільки поїздок виконано за певний період (з 1 по 3 травня)?
2 SELECT COUNT(ride_id) AS rides_in_period
3 FROM rides
4 WHERE ride_date BETWEEN '2026-05-01' AND '2026-05-03 23:59:59';
```

	rides_in_period bigint
1	6

Рисунок 6.10. Результат запиту з використанням оператора BETWEEN.

7. Завантаження результатів на репозиторій на GitHub.



Висновки:

У результаті виконання даної лабораторної роботи було успішно спроектовано та реалізовано реляційну базу даних для предметної області «Служба таксі» (Варіант №19). Під час роботи було створено концептуальну модель у вигляді ER-діаграми, розроблено SQL-скрипти для створення бази даних, налаштування трьох таблиць (clients, drivers, rides) із забезпеченням цілісності даних за допомогою первинних і зовнішніх ключів, а також створено ролі користувачів із різними правами доступу. Базу даних було наповнено тестовими даними, на основі яких успішно виконано 10 комплексних SQL-запитів (SELECT). Запити повністю розкрили бізнес-логіку варіанту та включали вибірку з умовами, сортування, групування, об'єднання таблиць (JOIN) та використання агрегатних функцій.

